Design Patterns Flashcards: Singleton & Factory

1. Singleton Pattern

Problem:

Ensure a class has only one instance and provide a global point of access to it.

Real-World Use Cases:

- Database Connection
- Logger
- Configuration Manager
- Cache Manager
- Auth Manager

JavaScript Example:

```
class Singleton {
  constructor() {
    if (Singleton.instance) return Singleton.instance;
    this.data = "I am the only instance";
    Singleton.instance = this;
  }
  showData() {
    console.log(this.data);
  }
}

const a = new Singleton();
const b = new Singleton();
console.log(a === b); // true
```

Interview Tip:

Singleton ensures data consistency and resource optimization when a single point of access is needed.

Design Patterns Flashcards: Singleton & Factory

2. Factory Pattern with Subclasses

Problem:

Creates objects without exposing the creation logic. Subclasses decide which class to instantiate.

Real-World Use Case:

Notification system (Email, SMS, Push)

JavaScript Example:

```
class Notification {
  send(msg) { throw new Error("Must be implemented"); }
}
class EmailNotification extends Notification {
  send(msg) { console.log(`EMAIL: ${msg}`); }
}
class SMSNotification extends Notification {
  send(msg) { console.log(`SMS: ${msg}`); }
}
class PushNotification extends Notification {
  send(msg) { console.log(`PUSH: ${msg}`); }
}
class NotificationFactory {
  createNotification(type) {
    switch(type) {
      case "email": return new EmailNotification();
      case "sms": return new SMSNotification();
      case "push": return new PushNotification();
      default: throw new Error("Invalid type");
    }
  }
}
```

Design Patterns Flashcards: Singleton & Factory

```
const factory = new NotificationFactory();
const notif = factory.createNotification("sms");
notif.send("Hello");
```

Interview Tip:

Promotes loose coupling and scalability. Easily extendable without modifying existing logic.